

Robot Communication Protocol

Industrial Robot Division - Software Department

Company	Suzhou Jiuwu Intelligent Technology Co., Ltd.
Department	Industrial Robot Division - Software Department
Date	2022-11-16
Version	Version 8 (8th Edition)
Description	Communication protocol for connecting and controlling robots
Confidentiality	Public / Open

Table of Contents

1. Overview
2. Data Packet Format
3. Connection Process
4. Examples
5. Commands

1. Overview

Robot communication is carried out via TCP. After establishing a connection to the robot using a Socket, communication is executed using the data packet format described in Chapter 2. All available commands, including their parameters and return values, are fully detailed in Chapter 5. This communication protocol can be utilized to develop custom client applications (similar to JunionViewer) to control and monitor the robots.

2. Data Packet Format

The entire communication data packet is structured into five distinct components: packet header, data length, delimiter, JSON data payload, and packet tail. The format layout is illustrated below:

Header	Length	Delimiter	Data Payload	Tail
\$#	length	##	{ . . . }	\$~
head	length	split	data (JSON)	tail

Component Specifications:

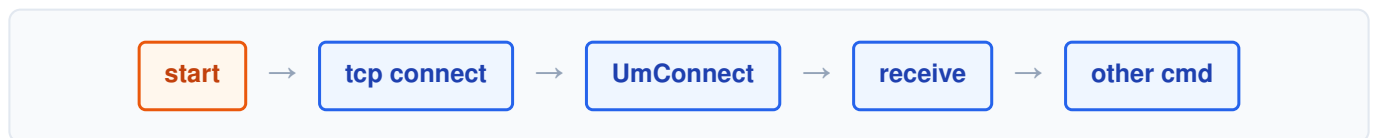
1. **Packet Header:** Consists of 2 bytes with the ASCII value \$# (hexadecimal 0x24 and 0x23).
2. **Length:** Specifies the size of the data payload located between the delimiter ## and the packet tail \$~. There is no hard limit on the number of bytes for this field, and the value is transmitted as a string.
 - *Example:* If the data block is 5 bytes long, the length field will be the ASCII string "5" (hexadecimal 0x35). If the length is 12 bytes, it will be the string "12" (hexadecimal 0x3132).
3. **Data:** A JSON-formatted string. Every valid data payload must contain two mandatory key-value pairs:
 - **#CMD#:** Command identifier indicating which specific command or action the data packet is intended to process (e.g., "UmConnect", "UmGetRobotInfo").
 - **#GAP#:** Time interval specified in milliseconds (ms). When a client requests telemetry or state data, certain attributes need to be retrieved periodically (e.g., coordinates). By configuring a positive interval in #GAP#, the

server will actively stream the data at that specified rate, eliminating the need for continuous polling requests from the client.

4. **Delimiter:** Consists of 2 bytes with the ASCII value ##. It serves to mark the end of the length string and the beginning of the JSON data payload.
5. **Packet Tail:** Consists of 2 bytes with the ASCII value \$~, marking the end of the complete packet.

3. Connection Process

The initialization and execution lifecycle of a session is described by the following workflow:



Once the Socket connection is successfully established, the client **must** first issue the "UmConnect" command to authenticate and retrieve the complete list of authorized/available commands. After receiving this list, it is highly recommended to immediately send all telemetry/information-collecting requests with an appropriate #GAP# interval configured. This allows the client to shift into a passive listening loop to receive and process state updates. Control or action commands are typically handled as discrete, one-time requests executed as needed.

4. Examples

Below is a step-by-step example demonstrating how to request and receive the robot's digital input (IO) data status:

1) Request Packet Example (Sending "UmGetInput")

Field Name	head	length	split	data	tail
Data	\$#	35	##	{"#CMD#": "UmGetInput", "#GAP#": 1000}	\$~
Description	Packet Header	Length of payload is 35	Delimiter	JSON Data Block	Packet Tail

Data Parsing Analysis:

- "#CMD#": "UmGetInput": Identifies that the request is targeting the input retrieval command.
- "#GAP#": 1000: Instructs the server to actively broadcast the input data block back to the client every 1000ms (1 second).

2) Response Packet Example (Receiving "UmGetInput" Data)

Field Name	head	length	split	data	tail
Data	\$#	44	##	{"#CMD#": "UmGetInput", "input": [0,0], "num": 2}	\$~
Description	Packet Header	Length of payload is 44	Delimiter	JSON Data Response	Packet Tail

Data Parsing Analysis:

- "#CMD#": "UmGetInput": Indicates the packet contains response data for the input request.
- "input": [0,0]: The actual requested data array, currently containing two elements initialized to 0.
- "num": 2: Represents the total number of elements present in the input array.

5. Commands

Commands to communicate with the robot correspond to the value of the #CMD# key within the JSON payload. All supported commands are detailed below:

1) UmConnect

Description:

Establishes a logical session with the robot. Must be sent immediately after successful socket connection. The robot validates credentials and returns all authorized commands.

Parameters:

user (string), password (string), device_type (string). Options for device_type include: "iphone", "android", "ipad", "pc", "local".

Return:

A list of all accessible commands, along with their parameters and return metadata. Format: commands: [{cmd(string), arg(string), ret(string)}, ...]

Example:

```
##87##{"#CMD#":"UmConnect", "#GAP#":-1, "user":"test", "password": "test",  
"device_type": "iphone"}$~
```

2) UmDrive

Description:

Direct teleoperation/driving controls for the robot. Expects linear velocity, angular velocity, and a scaling speed factor.

Parameters:

Linear velocity (trans (int), in mm/s); rotational velocity (rot (int), in °/s); and speed scale factor (speed (int), range [0, 100]). The final output velocities executed by the robot are computed as: linear velocity = $trans \times speed / 100$, angular velocity = $rot \times speed / 100$.

Return: None

Example:

```
##62##{"#CMD#":"UmDrive", "#GAP#":-1, "trans":300, "rot":30, "speed":10}$~
```

3) UmDock

Description:

Commands the robot to suspend current actions and navigate to its automatic charging dock.

Parameters: None

Return: None

Example:

```
$#29##{"#CMD#": "UmDock", "#GAP#": -1}$~
```

4) UmGetCurTask

Description: Retrieves detailed information regarding the robot's currently active task or mission.

Parameters: None

Return:

Data is nested within a data sub-JSON object containing 6 string elements: active route name (routes), current step key (key), current task definition (value), current ID (id), execution mode (mode), and execution status (status). Structure: data: {routes(string), key(string), value(string), id(string), mode(string), status(string)}.

Example:

```
$#xxx##{"#CMD#": "UmGetCurTask", "#GAP#": -1, "data": {"id": "", "key": "d111",  
"mode": "Stop", "routes": "Samples", "status": "Stopped", "value": {"cmd": "return",  
"comment": "mode wait for 1s", "flag": "1"}}}$~
```

5) UmGetInput

Description:

Retrieves the hardware digital input (IO) state. Returns the total byte count and an array representation of the bit states.

Parameters: None

Return:

num (int) and input (array). num is the element count of the array. The input value is an array of integers where the lower 8 bits of each element correspond to input states. The first element contains the highest 8 bits of the full IO structure, and the last element contains the lowest 8 bits. Structure: input: [byte1(int), byte2(int)...].

Technical Explanation: If num is 2, there are 2 elements in the array (16 bits total). If the returned array is [1, 2], the binary values are 00000001 and 00000010, combined to form 00000001 00000010. In this specific state, the digital input bits at position 2 and position 9 are high (1), and all others are low (0).

Example:

```
$#35##{"#CMD#":"UmGetInput", "#GAP#":1000}$~
```

6) UmGetLaser

Description:

Retrieves raw distance profiles from the on-board LiDAR sensors. Supports configurations with multiple sensors.

Parameters: None

Return:

Returns an array of objects corresponding to each active laser scanner. Each object contains the count of data points num (representing X/Y coordinate pairs) and a space-delimited string points containing the actual sequence of coordinate values. Structure: data(array): [{num(int), points: "x1 y1 x2 y2..."}, ...].

Example:

```
$#34##{"#CMD#":"UmGetLaser", "#GAP#":200}$~
```

7) UmGetLocState

Description: Retrieves the current localization confidence metric from the navigation sub-system.

Parameters: None

Return: score (int) representing the quality/confidence of the map matching alignment.

Example:

```
##37##{"#CMD#":"UmGetLocState", "#GAP#":200}$~
```

8) UmGetMap

Description:

Retrieves complete spatial map records including geometric boundaries and zoning metadata.

Parameters: None

Return:

Returns a structured object comprising a Header, resolution (MapRes in mm/pixel), coordinate bounds (MaxPose, MinPose), counts (NumLines, NumPoints), functional zoning regions (Objs), obstacle line segments (ObsLines), and discretized obstacle points (ObsPoints).

- Functional areas are arrays of objects, e.g., "ForbiddenArea":[{"name": "", "points": "-100 -200 300 400", "pose":"000"}].
- Obstacle lines are strings of space-separated coordinates defining segment ends: "x1 y1 x2 y2".
- Obstacle points are compressed in a 2D array format where the first item of each sub-array denotes an X coordinate, followed by all valid obstacle Y values on that column: [[x1, y11, y12...], [x2, y21, y22...]].

Example:

```
##31##{"#CMD#":"UmGetMap", "#GAP#":-1}$~
```

9) UmGetMapName

Description:

Retrieves the identifier/file name of the map currently active within the robot's localization system.

Parameters: None

Return: name (string)

Example:

```
##37##{"#CMD#":"UmGetMapName", "#GAP#":2000}$~
```

10) UmGetMotorState

Description: Retrieves the current excitation/enable state of the primary mobility motor drivers.

Parameters: None

Return: flag (bool) indicating whether the motors are energized and active.

Example:

```
##40##{"#CMD#":"UmGetMotorState", "#GAP#":1000}$~
```

11) UmGetName

Description: Retrieves the distinct network hostname or broadcast name assigned to the robot.

Parameters: None

Return: name (string)

Example:

```
##32##{"#CMD#":"UmGetName", "#GAP#":-1}$~
```

12) UmGetOutput

Description:

Retrieves the current state configurations of the digital output channels. Matches the architectural format of the input subsystem.

Parameters: None

Return:

num (int): size of the output array in bytes; output (array): [byte1(int), byte2(int)...].

Example:

```
##36##{"#CMD#":"UmGetOutput", "#GAP#":1000}$~
```

13) UmGetPath

Description: Retrieves the global trajectory path currently planned by the navigation pathfinder.

Parameters: None

Return:

Comprises the total waypoint count num (int) and an ordered array of coordinate pairs points: [{x, y}, ...].

Example:

```
##33##{"#CMD#":"UmGetPath", "#GAP#":200}$~
```

14) UmGetRobotInfo

Description:

Primary core state monitoring query. Fetches location, velocities, system status, and energy metrics simultaneously.

Parameters: None

Return:

Pose coordinates (x (int), y (int), th (int)); linear and rotational velocities (vel_f (double), vel_r (double)); task execution state indicators (mode (string), status (string)); and state of charge (battery (int) percentage).

Example:

```
##38##{"#CMD#":"UmGetRobotInfo", "#GAP#":200}$~
```

15) UmGetRobotSize

Description:

Retrieves the physical structural dimensions and kinematic center configurations of the chassis.

Parameters: None

Return:

Total structural width (width (double)), total structural length (length (double)), and distance offset from kinematic center to the leading edge (lengthfront (double)).

Example:

```
##37##{"#CMD#":"UmGetRobotSize", "#GAP#":-1}$~
```

16) UmGetRoutes

Description: Queries the complete registry of pre-programmed operational paths and motion macros.

Parameters: None

Return:

A data wrapper enclosing a routes array. Each element contains the profile name and internal structural definitions: {"name": "...", "content":{"a":"cmd", "a1":""...}}.

Example:

```
##34##{"#CMD#":"UmGetRoutes", "#GAP#":-1}$~
```

17) UmGetSafeDrive

Description:

Checks whether autonomous safety monitoring (collision avoidance scaling) is currently active.

Parameters: None

Return: flag (bool)

Example:

```
##39##{"#CMD#":"UmGetSafeDrive", "#GAP#":2000}$~
```

18) UmGoto

Description:

Commands the robot to initiate navigation toward a specific named target point or precise global coordinate.

Parameters: target (string): "goal" or "pose".
 goal (string): Target point name (used if target is "goal").
 poseX (int): Target coordinate X value in mm (used if target is "pose").
 poseY (int): Target coordinate Y value in mm (used if target is "pose").
 poseTh (int): Target orientation heading in degrees (used if target is "pose").

Return: None

Example:

```
##98##{"#CMD#": "UmGoto", "#GAP#": -1, "goal": "none", "poseTh": -23, "poseX": 5686, "poseY": 2602, "target": "pose"}$~
```

19) UmIdle

Description:

Forces the robot's state machine to abort current goals and transition immediately into a passive Idle state.

Parameters: None

Return: None

Example:

```
##29##{"#CMD#": "UmIdle", "#GAP#": -1}$~
```

20) UmLocalize

Description:

Overrides the current localization pose estimation, anchoring the robot to a known landmark/point or explicit coordinates. Parameters mirror UmGoto.

Parameters:

target (string): "goal" or "pose".

goal (string): Landmark name on the map.

poseX (int), poseY (int), poseTh (int): Target X (mm), Y (mm), and orientation angle (deg, valid range: (-180, 180]).

Return: None

Example:

```
##96##{"#CMD#":"UmLocalize", "#GAP#":-1, "goal":"","poseTh": 0, "poseX":3513, "poseY":4189, "target":"pose"}$~
```

21) UmRoutes

Description: Triggers the execution of a preloaded motion route or task profile.

Parameters:

Route identifier name routes (string) and the entry block step key (string). An execution instance id (string) can be optionally supplied.

Return: None

Example:

```
##60##{"#CMD#":"UmRoutes", "#GAP#":-1, "key":"d", "routes":"Samples"}$~
```

22) UmSchedulerList

Description: Enqueues a sequential list of multi-route tasks to be processed by the internal job scheduler.

Parameters:

Total route elements size (int) and an array of task items `list` (array) using the specific notation format "routes:key". Structure: `size(int), list(array):["routes1:key1", "routes2:key2"...]`.

Return: None

Example:

```
$#__##{"#CMD#":"UmSchedulerList","size":1,"list":["Samples:a"]}~
```

23) UmSchedulerThis

Description:

Injects an ad-hoc, temporary route definition directly into the runtime buffer for immediate execution without saving to disk.

Parameters:

The full inline structural content (`content`), route label (`name`), initial sequence entry element (`key`), and optional sequence id.

Return: None

Example:

```
$#__##{"#CMD#":"UmSchedulerThis","name":"Samples","key":"a","content":{"a":{"cmd":"wait","time":-1}}}}~
```

24) UmSetMotor

Description:

Enables or disables (energizes/de-energizes) the primary drive system actuators. Useful for physical recovery or manual pushing.

Parameters: `flag (bool)`: true to enable/lock drivers, false to release/disable.

Return: None

25) UmSetOutput

Description: Sets the full byte register mapping of the digital hardware outputs.

Parameters:

Register size `length` (int) – currently restricted to 2 (16 bits total); integer configurations for high 8 bits (`high` (int)) and low 8 bits (`low` (int)).

Return: None

26) UmSetOutputByte

Description: Modifies a solitary discrete bit channel on the digital output register block.

Parameters:

Target bit identifier `index num` (int), restricted to range 1~16; target value `flag` (int): 0 (low) or 1 (high).

Return: None

27) UmSetSafeDrive

Description: Toggles runtime active obstacle proximity deceleration scaling and safety parameters.

Parameters: `flag` (bool)

Return: None

28) UmSetVolume

Description: Adjusts the audio amplification output level of the on-board notification speaker systems.

Parameters: Target level value `volume` (int), valid range 0~100.

Return: None

29) UmStop

Description:

Abruptly issues a priority halt to all active kinematic movement mechanisms, putting the robot into a Stopped mode.

Parameters: None

Return: None